REFLECTED XSS - HTML CONTEXT

TryHackMe Learning & Practical Report

Prepared by	Belal Rabea Khalifa
Program	IqraSec Academy, WAPT Internship (Cohort 02), Phase 1
Task	4. Common Web Vulnerabilities - Cross-Site Scripting (XSS)
Platform	PortSwigger Web Security Academy
Lab	Reflected XSS into HTML context with nothing encoded (Apprentice)
Link	portswigger.net/web-security/cross-site-scripting/reflected
Focus	reflected XSS, search parameter, script execution
Status	Solved
Date	October 2026

1. Objective

Solve the PortSwigger lab "Reflected XSS into HTML context with nothing encoded": inject a script into the search feature so it runs in the browser.

2. Concept / Understanding

- **Reflected XSS:** happens when the application takes input (like a search term) and puts it back into the page's HTML without encoding it. The browser then reads the input as real HTML/JavaScript instead of plain text.
- **Payload used:** `<script>alert(1)</script>`. If the application reflects this as-is inside the HTML body, the browser runs it as a script tag and executes alert(1).
- This is called *reflected* because the payload comes back in the same response, from a URL or a form the victim submits (or is tricked into opening). It is different from stored XSS, where the payload is saved on the server and runs for every visitor.
- A popup from alert(1) is the standard, safe way to prove script execution in a lab; in a real attack the same spot could run code to steal cookies, tokens, or perform actions as the victim.

3. Practical Work

Lab: Reflected XSS into HTML context with nothing encoded. I did the following steps:

#	Step	What I did / found
1	Opened the blog's search box.	Found the search feature that reflects the search term back on the page.
2	Entered the payload <script>alert(1)</script> and clicked Search.	The URL became ?search=<script>alert(1)</script> (URL-encoded).
3	The page loaded.	The script ran immediately: a popup box appeared showing "1" from the site's own domain, proving the script executed in the page's context.
4	Closed the popup (OK) and checked the page.	The page showed "0 search results for '' and the lab status changed to Solved.

4. Tools Used

- Web browser (search box and address bar)

- PortSwigger Web Security Academy lab

5. Evidence

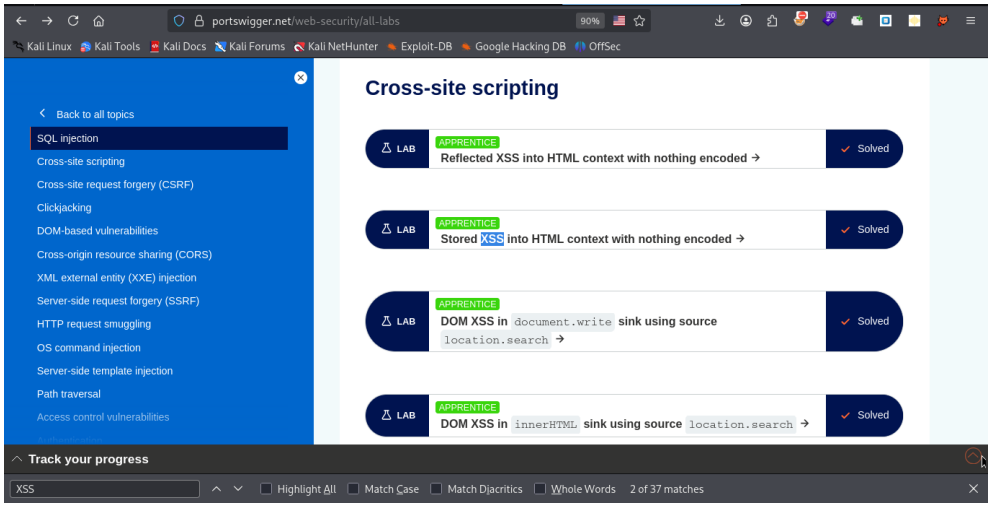


Figure 1: the Cross-site scripting labs list. "Reflected XSS into HTML context with nothing encoded", marked Solved.

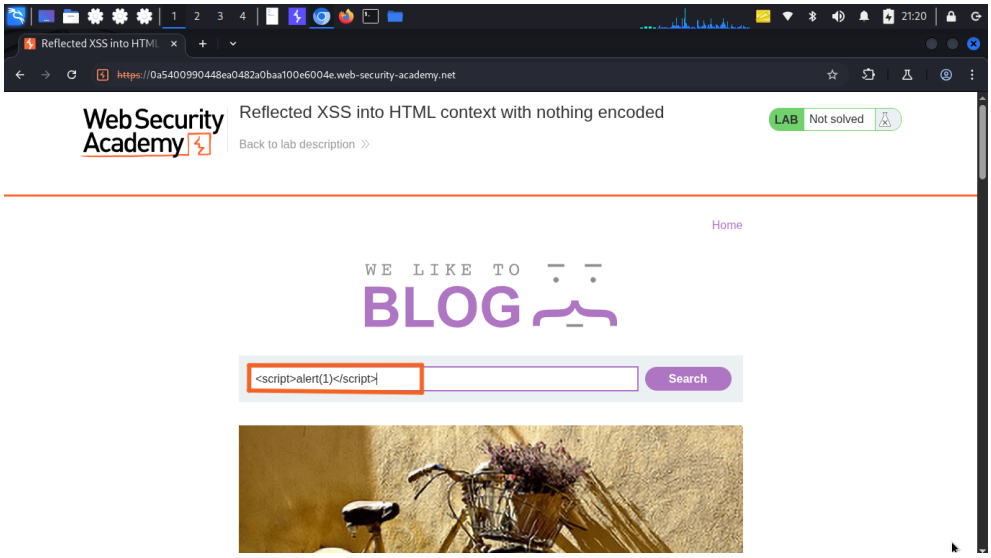


Figure 2: the payload alert(1) typed into the search box, before submitting (status: Not solved).

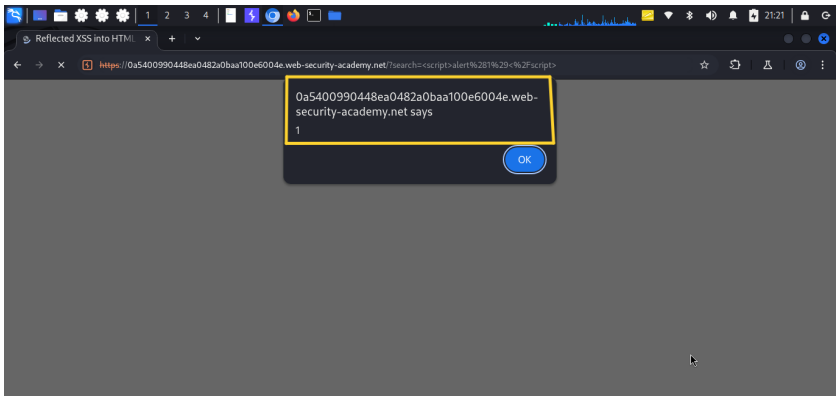


Figure 3: the alert(1) popup triggered by the injected script, shown by the site's own domain.

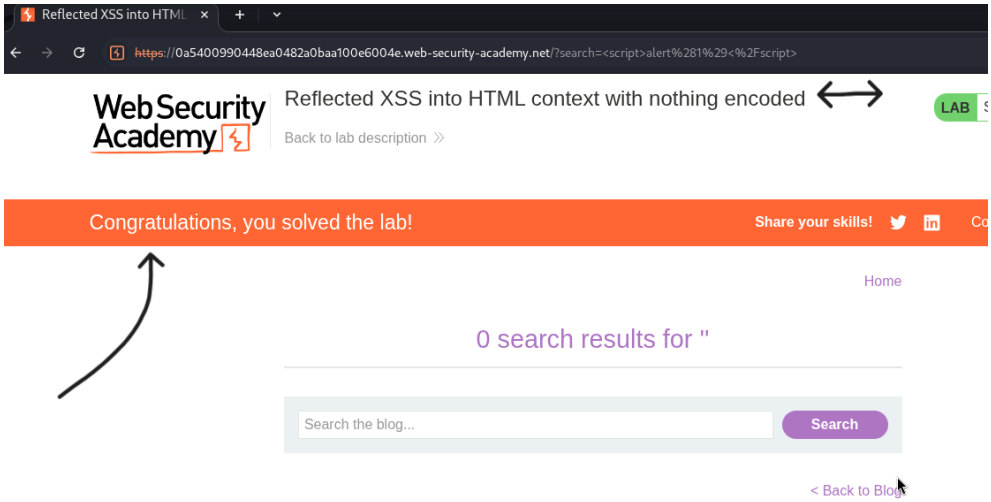


Figure 4: "Congratulations, you solved the lab!", with the payload visible in the URL.

6. Result

Solved the lab. The unencoded alert(1) payload was reflected into the page's HTML and executed by the browser.

7. Key Learnings

- Reflected XSS happens when user input is put back into the HTML response without encoding.
- A simple alert(1) payload is enough to prove the vulnerability when nothing is encoded.
- The payload only runs when the victim opens the crafted URL or submits the form, unlike stored XSS which runs for every visitor automatically.
- The fix is always to encode output based on where it is placed in the page, not just to block a few tags or characters.

8. Impact and Remediation

Impact: an attacker can run JavaScript in a victim's browser, under the real site's domain. This can steal session cookies or tokens, perform actions as the victim, or redirect them to a malicious page, by sending them a crafted link.

Remediation:

- Encode all user input before reflecting it in HTML (turn <, >, &, ", ' into their HTML-safe entities).
- Apply a Content Security Policy (CSP) header to limit which scripts the page is allowed to run.
- Validate input, but treat output encoding as the main defense, since input validation alone cannot stop every case.